Templates and Forms in Django

Templating, Handle form data

Simone Ricci

Modify a View Class to extend functionality

Override the inherited method

The best way to add or modify the behaviour of a class is to **override** the method defined by the Base classes.

ListView

class **django.views.generic.list.ListView**

A page representing a list of objects.

While this view is executing, **self.object_list** will contain the list of objects (usually, but not necessarily a queryset) that the view is operating upon.

Ancestors (MRO)

This view inherits methods and attributes from the following views:

- **django.views.generic.list.MultipleObjectTemplateResponseMixin**
- **django.views.generic.base.TemplateResponseMixin**
- **django.views.generic.list.BaseListView**
- **django.views.generic.list.MultipleObjectMixin**
- **django.views.generic.base.View**

override!

```
class ArticleListView(ListView):
    model = Article
    template_name = "article_list.html"

    def get_queryset(self):
        return Article.objects.all().filter(author=self.request.user)

    def get(self, request: HttpRequest, *args: Any, **kwargs: Any) -> HttpResponse:
        # If the user is not logged in, redirect to signup page.
        if not request.user.is_authenticated:
            return redirect("home")
            # return redirect("signup")
        return super().get(request, *args, **kwargs)
```

Override the inherited method

CreateView

class **django.views.generic.edit.CreateView**

A view that displays a form for creating an object, redisplaying the form with validation errors (if there are any) and saving the object.

Ancestors (MRO)

This view inherits methods and attributes from the following views:

- `django.views.generic.detail.SingleObjectTemplateResponseMixin`
- `django.views.generic.base.TemplateResponseMixin`
- `django.views.generic.edit.BaseCreateView`
- `django.views.generic.edit.ModelFormMixin`
- `django.views.generic.edit.FormMixin`
- `django.views.generic.detail.SingleObjectMixin`
- `django.views.generic.edit.ProcessFormView`
- `django.views.generic.base.View`

override!

```
class ArticleCreateView(CreateView):  # new
    model = Article
    template_name = "article_new.html"
    fields = (
        "title",
        "body"
    )
    def post(self, request: HttpRequest, *args: str, **kwargs: Any) -> HttpResponse:
        form = self.get_form()
        if form.is_valid():
            # Saving with commit=False gets you a model object, then you can add your extra data and save it.
            article = form.save(commit=False)
            article.author = self.request.user
            article.save()
            return self.form_valid(form)
        else:
            return self.form_invalid(form)
```

Templates

Templates

A **Template system** allows to define the structure and layout of web pages.

It **separates** the **presentation logic** from **the business logic** and helps in creating reusable components.

Template files use a special syntax to incorporate **dynamic content**, such as variables, loops, and conditionals, which are replaced with actual values at **runtime**.

Templates can be used to generate HTML, XML, CSV, and other formats.

Django framework uses **Template engines** to render them. The default is the **Django Template Language** (**DTL**). A Django project can be configured with one or several template engines (or even zero if you don't use templates).

Other template languages may be available from third-parties like **Jinja2**. You can also write your own custom Template Language.

- By default, Django's template loader will look within each app for related **/templates**
- Create a **single project-level templates directory** and place all templates within there. Update **settings.py** file to tell Django where to look for templates.

Django Template Language (DTL)

A Django template is a text document or a Python string marked-up using the Django template language.

Some constructs are recognized and interpreted by the **template engine**. The main ones are **variables** and **tags**.

A template is rendered with a **context**! Rendering replaces variables with their values, which are looked up in the context, and executes tags.

Everything else is output as is! (tipically HTML)

The syntax of the Django template language involves four constructs:

- **Variables**
- **Tags**
- **Filters**
- **Comments**

There is also **template inheritance**. It allows you to create a base template that can be extended by other templates (for a consistent look and feel).

Variables

A variable outputs a value from the **context**, which is a dict-like object mapping keys to values.

Variables are surrounded by double curly braces: **{{ variable }}**.

When a template is rendered, the variable is replaced with its actual value.

```
My first name is {{ first_name }}. My last name is {{ last_name }}.
```

With a context of {'first_name': 'John', 'last_name': 'Doe'}

```
My first name is John. My last name is Doe.
```

```
{{ my_dict.key }}
{{ my_object.attribute }}
{{ my_list.0 }}
```

Use a dot **.** to access attributes of a variable.

If you use a variable that doesn't exist, the template system will insert the value of the string_if_invalid option, which is set to " " (the empty string) by default.

Variable attributes that begin with an underscore may not be accessed as they're generally considered **private**.

Variables lookups

Technically, when the template system encounters a **dot**, it tries the following lookups, in this order:

1) Dictionary lookup
2) Attribute or method lookup
3) Numeric index lookup

If the resulting value is callable, it is called with **no arguments**. The result of the call becomes the template value.

This lookup order can cause some unexpected behavior.

For example, a loop over a collections.defaultdict:

```
{% for k, v in defaultdict.items %}
    Do something with k and v here...
{% endfor %}
```

Because dictionary lookup happens first, that behavior kicks in and **provides a default value** instead of using the intended .items() method.

Variables Lookups Examples

```
from collections import defaultdict
d = defaultdict(lambda: "DEFAULT")
# d has no 'items' key in its internal dict, so d['items'] → "DEFAULT"
```

You might expect it to call the built-in **.items()** method, returning a list of key/value pairs.

```
{% for key, val in d.items %}
   {{ key }} → {{ val }}
{% endfor %}
```

But because mapping lookup comes first, the engine does:

Try d['items'] → this does exist and **defaultdict** returns "DEFAULT".

It treats that result ("DEFAULT") as the lookup result.

Since "DEFAULT" isn't callable, it stops right there.

You never get to step 2's "attribute lookup" of the real d.items method, so you end up iterating over a **string**!

Tags

Tags provide **arbitrary logic** in the rendering process.

Tags are surrounded by curly braces and percent signs, like this: **{% *tag* %}**.

This definition is deliberately vague: A tag can output content, serve as a control structure e.g. an “if” statement or a “for” loop, grab content from a database, or even enable access to other template tags.

Most tags accept arguments:

```
{% cycle 'odd' 'even' %}
```

Some tags require beginning and ending tags:

```
{% if user.is_authenticated %}Hello, {{ user.username }}.{% endif %}
```

```
{% extends "base.html" %}
{% load article_extras %}
```

Built-in template tags

- **block**: Defines a block that can be overridden by child templates (**inheritance**)
- **comment**: Ignores everything between {% comment %} and {% endcomment %}
- **csrf_token:** is used for CSRF protection (generates a hidden input field that contains a unique token)
- **cycle**: Produces one of its arguments in order each time this tag is encountered
- **for:** Loops over each item in an array, making the item available in a context variable

```
<ul>
{% for athlete in athlete_list %}
    <li>{{ athlete.name }}</li>
{% endfor %}
</ul>
```

```
{% if athlete_list %}
    Number of athletes: {{ athlete_list|length }}
{% elif athlete_in_locker_room_list %}
    Athletes should be out of the locker room soon!
{% else %}
    No athletes.
{% endif %}
```

- **if:** evaluates a variable, and if that variable is "*true*" (i.e. exists, is not empty, and is not a false boolean value) the contents of the block are output
- **extends:** Signals that this template extends a parent template
- **include:** Loads a template and renders it with the current context
- **url**: Returns an absolute path reference (a URL without the domain name) matching a given view and ***optional parameters***. The first argument is a **URL pattern name**.

```
{% url 'some-url-name' v1 v2 %}
```

Filters

Filters **transform** the values of **variables** and **tag** arguments.

Filters are used to modify the output of a variable, such as converting it to uppercase or formatting it as a date.

Filters are applied to a variable using a **pipe character**, like this: **{{ variable | filter }}**.

Built-in filters:

- **add**: Adds the argument to the value. `{{ value|add:"2" }}`
- **date**: Formats a date according to the given format. `{{ my_date|date:"Y-m-d" }}`
- **default**: If value evaluates to False, uses the given default. Otherwise, uses the value. `{{ value|default:"nothing" }}`
- **dictsort**: Takes a list of dictionaries and returns that list sorted by the key given in the argument. `{{ value|dictsort:"name" }}`
- **filesizeformat**: Formats the value like a '*human-readable*' file size (i.e. '13 KB', '4.1 MB', '102 bytes', etc.).
- **first**: Returns the first item in a list.

Creating custom tags and filters

Django provides the ability to create custom template **tags** and **filters** to extend the functionality of the built-in template language.

Custom tags and filters can be used to perform complex logic, process data, or generate dynamic content.

The **@register.simple_tag decorator** to register the function as a simple tag.

The **@register.filter decorator** to register the function as a simple filter.

```
from django import template

register = template.Library()

@register.simple_tag
def my_tag():
    return "This is my custom tag."
```

```
from django import template

register = template.Library()

@register.filter
def my_filter(value):
    return value.upper()
```

```
{% load my_tags %}

<p>{% my_tag %}</p>
```

```
{% load my_filters %}

<p>{{ my_string|my_filter }}</p>
```

Creating custom tags and filters

After extending the template engine with custom tags and filters using Python, it is possible to make them available to templates using the **{% load %}** tag.

The most common place to specify custom template tags and filters is **inside a Django app**.

When a Django app is added to **INSTALLED_APPS**, any tags it defines in the conventional location are automatically made available to load within templates.

The app should contain a **templatetags** directory, at the same level as models.py, views.py, etc.

If this doesn't already exist, create it.

```
polls/
    __init__.py
    models.py
    templatetags/
        __init__.py
        poll_extras.py
    views.py
```

!

```
{% load poll_extras %}
```

Creating custom tags and filters

To be a valid tag library, the module must contain a module-level variable named **register** that is a **template.Library** instance, in which all the tags and filters are registered.

Any exception raised from a template filter will be exposed as a server error. Thus, filter functions should avoid raising exceptions if there is a reasonable fallback value to return.

Custom filters take one or two arguments:

- The value of the **variable** (input) – not necessarily a string.
- The value of the **argument** – this can have a default value.

For template filter that only **expects a string** as the first argument, you should use the decorator **stringfilter**. This will **convert an object to its string value**.

```
from django import template
from django.template.defaultfilters import stringfilter

register = template.Library()


@register.filter
@stringfilter
def lower(value):
    return value.lower()
```

Variables Lookups .items() Possible Solution

Write a tiny custom template filter.

```python
# templatetags/my_extras.py
from django import template
register = template.Library()

@register.filter
def call_items(d):
    return d.items()
```

Use in the template file like this.

```
{% load my_extras %}
{% for key, val in d|call_items %}
  {{ key }} → {{ val }}
{% endfor %}
```

This will sidestep the “defaultdict provides its own value for missing keys” behavior and give you the real **.items()** output.

Template inheritance

Template inheritance allows you to build a **base "skeleton" template** that contains all the common elements of your site and defines **blocks** that child templates can override.

base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="stylesheet" href="style.css">
    <title>{% block title %}My amazing site{% endblock %}</title>
</head>

<body>
    <div id="sidebar">
        {% block sidebar %}
        <ul>
            <li><a href="/">Home</a></li>
            <li><a href="/blog/">Blog</a></li>
        </ul>
        {% endblock %}
    </div>

    <div id="content">
        {% block content %}{% endblock %}
    </div>
</body>
</html>
```

child.html

```
{% extends "base.html" %}

{% block title %}My amazing blog{% endblock %}

{% block content %}
{% for entry in blog_entries %}
    <h2>{{ entry.title }}</h2>
    <p>{{ entry.body }}</p>
{% endfor %}
{% endblock %}
```

HTML generated

```
<!DOCTYPE html>
<html lang="en">
<head>
    <link rel="stylesheet" href="style.css">
    <title>My amazing blog</title>
</head>

<body>
    <div id="sidebar">
        <ul>
            <li><a href="/">Home</a></li>
            <li><a href="/blog/">Blog</a></li>
        </ul>
    </div>

    <div id="content">
        <h2>Entry one</h2>
        <p>This is my first entry.</p>

        <h2>Entry two</h2>
        <p>This is my second entry.</p>
    </div>
</body>
</html>
```

Template inheritance

If you use **{% extends %}** in a template, it **must be the first template tag** in that template. Template inheritance won't work, otherwise!

The **block** tag does is to tell the template engine that a child template may override those portions of the template.

When the template system evaluates the child, **first it locates the parent**.

If the child template *didn't define a block*, the value from the parent template is used. Content within a **{% block %}** tag in a parent template is always **used as a fallback**.

One common way of using inheritance is the following **three-level approach**:

1. Create a **base.html** template that holds the main look-and-feel of your site.
2. Create a **base_SECTIONNAME.html** template for each "section" of your site. For example, base_news.html, base_sports.html. These templates all extend **base.html** and include section-specific styles/design.
3. Create individual templates for each type of page, such as a news article or blog entry. These templates extend the appropriate section template.

Template inheritance

Inheritance maximizes code reuse and helps to add items to shared content areas, such as section-wide navigation.

More **{% block %}** tags in your base templates are better. Remember, child templates don't have to define all parent blocks.

If you find yourself duplicating content in a number of templates, it probably means you should move that content to a **{% block %}** in a parent template.

Variables created outside of a **{% block %}** using the template tag **as** syntax can't be used inside the block.

```
{% translate "Title" as title %}
{% block content %}{{ title }}{% endblock %}
```

{% block %} tags are evaluated first. That's why the content of a block is always overridden, regardless of the truthiness of surrounding tags.

```
{% if change_title %}
    {% block title %}Hello!{% endblock title %}
{% endif %}
```

Conditionally override

If you really want only override when change_title is True, otherwise leave the parent's title intact, flip your logic so the **{% if %}** lives inside the block, not the other way around.

```
{% block title %}
  {% if change_title %}
    Hello!
  {% else %}
    {{ block.super }}   {# renders parent block's content #}
  {% endif %}
{% endblock title %}
```

At render time you choose whether to spit out Hello! or hand back to **block.super**.

It is different from previous

```
{% if change_title %}
    {% block title %}Hello!{% endblock title %}
{% endif %}
```

Django Template Rendering Process

Django looks in each directory in your **TEMPLATES[...]['DIRS']** and each app's **templates/** folder. If a cached (compiled) version exists, it's reused.

1

```
from django.template import loader

# Find and compile "app/page.html"
template = loader.get_template("app/page.html")
```

Build the context:

2

```
# Your view-specific values
my_data = {"foo": "bar", "items": [1,2,3]}

# Wrap in a RequestContext to apply context processors
from django.template import RequestContext

# If you have the request object, pass it in so processors
context = RequestContext(request, my_data)
```

3

```
# Render the node tree into an HTML string
rendered_html = template.render(context)
```

Template Render:

Variable nodes ({{ foo }}) get looked up in context (and filtered). Tag nodes ({% for %}, {% url %}, {% block %}…) execute their logic and recurse. Builds one large Unicode string of HTML.

4

```
from django.http import HttpResponse

response = HttpResponse(
    content=rendered_html,
    content_type="text/html; charset=utf-8",
    status=200,
)
return response
```

Django shortcut

The package **django.shortcuts** collects helper functions and classes that **"span" multiple levels of MVT.**

render(request, template_name, context=None, content_type=None, status=None, using=None)

Combines a given template with a given context dictionary and returns an HttpResponse object with that rendered text.

```
from django.shortcuts import render

def home(request):
    context = {'name': 'John'}
    return render(request, 'home.html', context)
```

redirect(to, *args, permanent=False, **kwargs)

Returns an HttpResponseRedirect to the appropriate URL for the arguments passed:

- A model: the model's get_absolute_url() function will be called.
- A view name, possibly with arguments: reverse() will be used to reverse-resolve the name.
- An absolute or relative URL, which will be used as-is for the redirect location.

```
def my_view(request):
    ...
    return redirect("some-view-name", foo="bar")
```

get_object_or_404(klass, *args, **kwargs)

Calls get() on a given model manager, but it raises Http404 instead of the model's DoesNotExist exception.

Django vs Jinja2

Variable Delimiters: Django: {{ variable }} vs Jinja2: {{ variable }}

Tag Delimiters: Django: {% tag %} vs Jinja2: {% tag %}

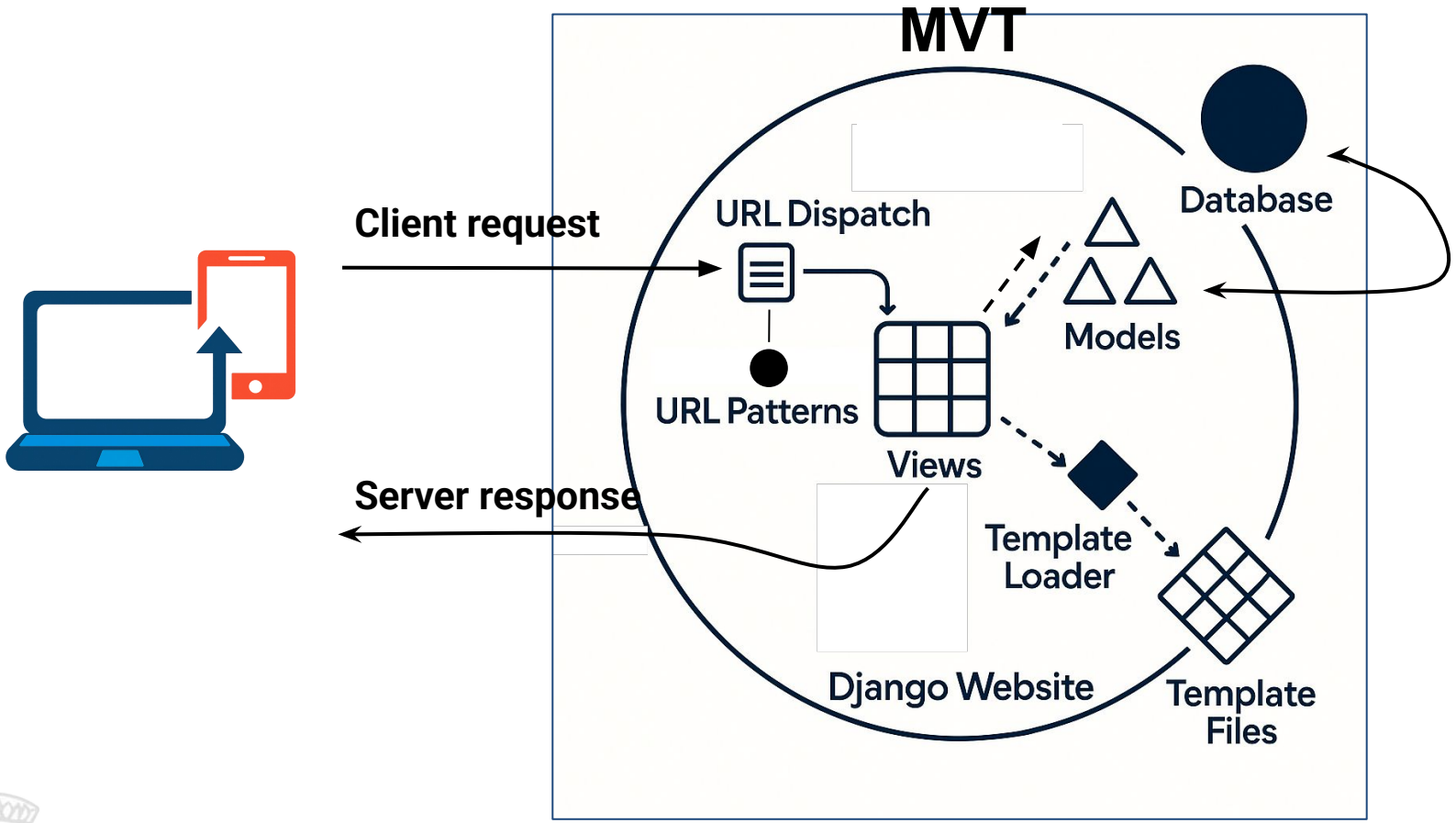
Filters: Django: {{ variable | filter }} vs Jinja2: {{ variable | filter }}

Jinja2 has variable assignment: {% **set** total = 42 %}; it supports whitespace trimming with **-** and **can call functions and methods** as long as they are passed into the context.

```
jinja2<br>{{ myfunc(arg1, arg2) }}<br>
```

Aspect	Django Template	Jinja2 Template
Flexibility	Limited by design	More flexible and powerful
Learning curve	Easier for beginners	Slightly more complex
Security	Strong default auto-escaping	Also secure, but more control
Framework integration	Tight with Django	More framework-agnostic

MVT Recap


MVT
Client request
URL Dispatch
Database
Models
URL Patterns
Views
Server response
Template Loader
Template Files
Django Website

Forms

HTML Form

A form is a collection of elements inside **<form>...</form>** that allow a visitor to do things like enter text, select options, …, and then **send that information back to the server**.

They are one of the more complicated and **error-prone** aspects of web development because any time you accept user input there are **security concerns**.

Some of these form interface elements (**<input>** elements) are built into HTML itself (**type="text", type="checkbox"**)

The form data are sent to the URL specified in the **<form>** **action=""** attribute.

As well as its **<input>** elements, a form must specify two things:

- where: the URL to which the data corresponding to the user's input should be returned
- how: the HTTP method the data should be returned by

When the **<input type="submit" value="Submit">** element is triggered, the data is returned to **URL**.

All forms must be **properly rendered**, **validated**, **and saved** to the database.

HTML Form

```
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
```

First name:

John

Last name:

Doe

Submit

Your input was received as:

fname=John&lname=Doe

GET and POST

GET and POST are the only HTTP methods to use when dealing with forms.

POST method: the browser bundles up the form data, encodes it for transmission, sends it to the server, and then receives back its response.

GET method: the browser bundles the submitted data **into a string**, and uses this to compose a URL. The URL contains the address where the data must be sent, as well as the data keys and values.

Any request that could be used to **change the state** of the system (request that makes changes in the database) **should** use **POST**.

GET should be used only for requests that **do not affect the state** of the system. It would be unsuitable for a password form, because **the password would appear in the URL**!!

POST, coupled with other protections like **Django's CSRF protection** offers more control over access.

GET is suitable for things like a **web search form** (bookmark, shared, or resubmitted)

Forms in Django

At the heart of this system of components is Django's **Form class**.

As Django Model class describes the logical structure of an object and its behavior; a Form class **describes a form and determines how it works and appears**.

In a similar way that a model class's fields are mapped to database fields, a **Form class's fields are mapped to HTML form <input> elements**.

A form's **Fields** are themselves classes! (as for the Model class)

They **manage form data and perform validation** when a form is submitted.

A **DateField** and a **FileField** handle very different kinds of data and have to do different things with it.

A form field is represented to a user in the browser as an HTML **"widget"** - a piece of user interface machinery (input, select, textarea, …).

A **Widget** is Django's representation of an HTML input element. It handles the rendering of the HTML, and the extraction of data from a GET/POST dictionary.

Each field type has an appropriate **default Widget** class, but these can be overridden as required.

Widget

Whenever you specify a field on a form, Django will use a default widget that is appropriate to the type of data that is to be displayed (Built-in Field classes).

```
class CommentForm(forms.Form):
    name = forms.CharField()
    url = forms.URLField()
    comment = forms.CharField(widget=forms.Textarea)
```

optional extra arguments

```
BIRTH_YEAR_CHOICES = ["1980", "1981", "1982"]
FAVORITE_COLORS_CHOICES = {
    "blue": "Blue",
    "green": "Green",
    "black": "Black",
}


class SimpleForm(forms.Form):
    birth_year = forms.DateField(
        widget=forms.SelectDateWidget(years=BIRTH_YEAR_CHOICES)
    )
    favorite_colors = forms.MultipleChoiceField(
        required=False,
        widget=forms.CheckboxSelectMultiple,
        choices=FAVORITE_COLORS_CHOICES,
    )
```

attrs lets you add HTML attributes like CSS classes, or ids

```
class MyForm(forms.Form):
    username = forms.CharField(widget=forms.TextInput(attrs={
        'class': 'form-control',
        'placeholder': 'Enter username'
    }))
    birthdate = forms.DateField(widget=forms.DateInput(attrs={
        'type': 'date'
    }))
```

Forms in Django

When rendering an **object** in Django the steps are:

1) get hold of it in the view (fetch it from the database, for example)
2) pass it to the **template context**
3) expand it to HTML markup using **template variables**

Rendering a form in a template involves nearly the same work as rendering any other kind of object.

A Model instance in a view is retrieve it from the database. A Form is typically **instantiated in the view**.

It can be **leaved empty or prepopulated**, for example with:

- data from a saved model instance
- data that obtained from other sources
- data received from a previous **HTML form submission**

The last of these cases makes it possible for users not just to read a website, but also to send information back to it.

Define form in Django

The starting point for building a form in Django is extending **Form** class.

```
forms.py ¶

from django import forms


class NameForm(forms.Form):
    your_name = forms.CharField(label="Your name", max_length=100)
```

This defines a **Form class** with a single **field** (**your_name**).

from template context

```
<form action="/your-name/" method="post">
    <label for="your_name">Your name: </label>
    <input id="your_name" type="text" name="your_name" value="{{ current_name }}">
    <input type="submit" value="OK">
</form>
```

A Form instance has an **is_valid()** method, which runs validation routines for all its fields. When this method is called, **if all fields contain valid data**, it will:

- return True
- place the form's data in its **cleaned_data** attribute.

Views handle form

Form data sent back to a Django website is processed by a view, generally the same view which published the form.

This allows us to **reuse some of the same logic**.

To handle the form we need to **instantiate it** in the view for the URL where we want it to be published.

Takes data from the POST request

Instanziate the form when there is a GET request

views.py ¶

```
from django.http import HttpResponseRedirect
from django.shortcuts import render

from .forms import NameForm


def get_name(request):
    # if this is a POST request we need to process the form data
    if request.method == "POST":
        # create a form instance and populate it with data from the request:
        form = NameForm(request.POST)
        # check whether it's valid:
        if form.is_valid():
            # process the data in form.cleaned_data as required
            # ...
            # redirect to a new URL:
            return HttpResponseRedirect("/thanks/")

    # if a GET (or any other method) we'll create a blank form
    else:
        form = NameForm()

    return render(request, "name.html", {"form": form})
```

Templating a form

All the form's fields and their attributes will be unpacked into HTML markup from that **{{ *form* }}** by Django's template language.

If a form is called ***form*** in the context, {{ *form* }} will render its <label> and <input> elements appropriately.

Form's output does not include the surrounding <form> tags, or the form's submit control!

```
<form action="/your-name/" method="post">
    {% csrf_token %}
    {{ form }}
    <input type="submit" value="Submit">
</form>
```

```
return render(request, "name.html", {"form": form})
```

There are other output options though for the <label>/<input> pairs:

- **{{ form.as_div }}** : render them wrapped in tags.
- **{{ form.as_table }}** : render them as table cells wrapped in <tr> tags.
- **{{ form.as_p }}** : render them wrapped in <p> tags.
- **{{ form.as_ul }}** : render them wrapped in <li> tags.

Formsets

Formset is a way to manage **multiple instances** of a form (usually for a model) on a **single** page.

A family and its members; a book and its chapters; a class and its students; a shopping basket and its contents. You could use a formset for your model to add as many as you'd like at one time.

Form

```
class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

```
from django.forms import formset_factory

ContactFormSet = formset_factory(ContactForm, extra=3)
```

FormSet

View

```
def contact(request):
    if request.method == 'POST':
        formset = ContactFormSet(request.POST)
        if formset.is_valid():
            # Process the formset data
    else:
        formset = ContactFormSet()
    return render(request, 'contact.html', {'formset': formset})
```

context

Template

```
<form method="post">
    {% csrf_token %}
    {{ formset.management_form }}
    {% for form in formset %}
        {{ form.as_p }}
    {% endfor %}
    <button type="submit">Submit</button>
</form>
```

FormSet allows to display and **handle multiple forms in a single view**, and process the submitted data for each form individually.

Creating forms from models

ModelForm class is a convenient way to create a form based on a model. A ModelForm is essentially a regular Form that is automatically generated **based on a model's fields**.

It includes additional methods and attributes for handling model-specific functionality, such as saving the form data to the database.

```
class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=200)
    publisher = models.CharField(max_length=200)
    pub_date = models.DateField()
```

```
from django.forms import ModelForm
from .models import Book

class BookForm(ModelForm):
    class Meta:
        model = Book
        fields = '__all__'
```

The **Meta** class specify that the form should be **based on the Book model** and that all fields should be included in the form (***"__all__"***).

In the view function:

```
if request.method == 'POST':
    form = BookForm(request.POST)
    if form.is_valid():
        form.save()
```

ModelForm save() method

Every ModelForm also has a **save()** method.

This method creates and saves a database object **from the data bound to the form**.

A subclass of ModelForm can accept an existing model instance as the keyword argument **instance**. **save()** will update that instance.

If an instance not supplied, **save()** will create a **new instance** of the specified model.

This **save()** method accepts an optional **commit** keyword argument.

If you call **save()** with **commit=False**, then it will return an object that hasn't yet been saved to the database.

This is useful if you want to do custom processing on the object before saving it, or if you want to use one of the specialized model saving options. **commit is True by default**.

Form handling with Class-based views

Form processing generally has 3 paths:

1. Initial **GET** (blank or pre-populated form)
2. **POST** with invalid data (typically redisplay form with errors)
3. **POST** with valid data (process the data and typically redirect)

A lot of repeated code! To help avoid this, Django provides a collection of **generic Class-based views for form processing**.

```
class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

```
class ContactView(FormView):
    template_name = 'contact.html'
    form_class = ContactForm
    success_url = reverse_lazy('success')
```

Model forms

The generic views will automatically create a **ModelForm**, so long as they can work out which model class to use:

- If the **model** attribute is given, that model class will be used.
- If **get_object()** returns an object, the class of that object will be used.
- If a **queryset** is given, the model for that queryset will be used.

Model form views provide a **form_valid()** implementation that saves the model automatically.

It's possible to override it to write a different logic.

You don't even need to provide a **success_url** for **CreateView** or **UpdateView** - they will use **get_absolute_url()** on the model object if available.

If you want to use a custom ModelForm (o add extra validation), set **form_class** on the view.

Mixin View

Django's built-in class-based views provide a lot of functionality, but some of it you may want to use separately.

Django provides a number of mixins that provide more discrete functionality. Template rendering is encapsulated in the **TemplateResponseMixin**.

Every built in view which returns a TemplateResponse will call the **render_to_response()** method that **TemplateResponseMixin** provides.

The **get()** method implemented by both TemplateView and DetailView calls **render_to_response()**.

render_to_response() itself calls **get_template_names()**, which by default will look up **template_name** on the class-based view.

Two other mixins (**SingleObjectTemplateResponseMixin** and **MultipleObjectTemplateResponseMixin**) override **get_template_names()** method to provide more flexible defaults when dealing with actual objects.

For istance, the first is used in DetailView and the second in ListView.

Crispy-forms

Crispy-forms is a third-party Django app that provides a simple and powerful way to style your Django forms.

It is built on top of Django's built-in forms library and allows you to easily control the layout and styling of your forms using a variety of built-in templates and layout objects.

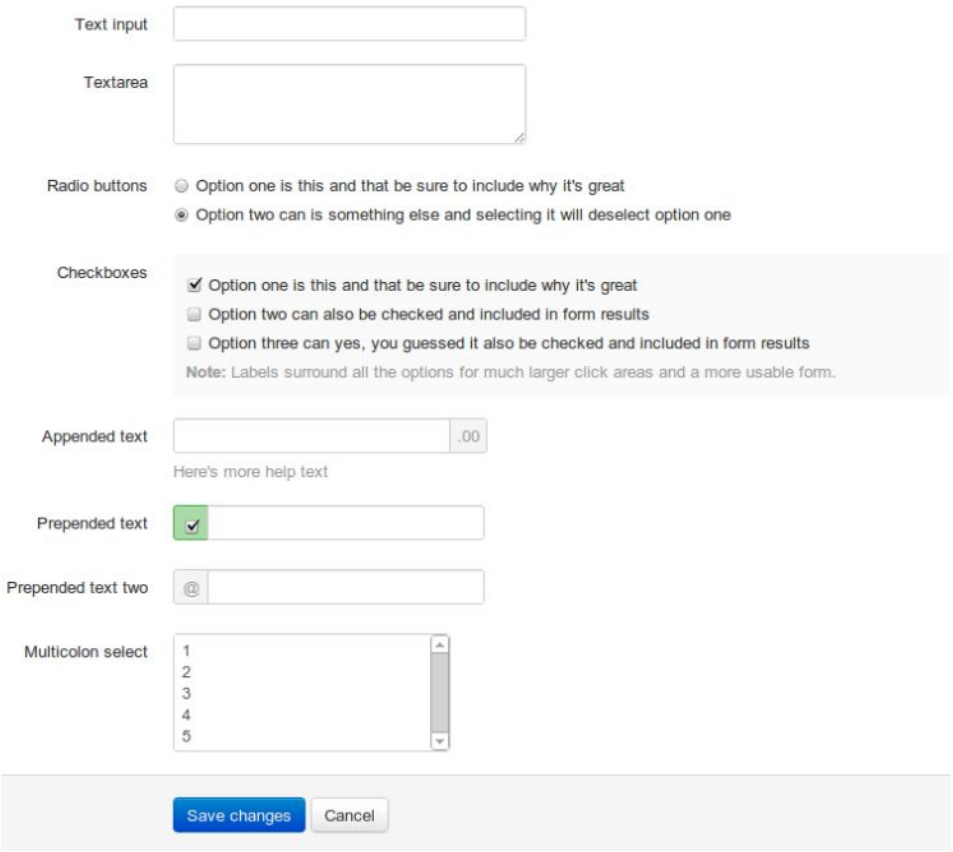
Install it: pip install django-crispy-forms

Add crispy-forms to your **INSTALLED_APPS** in your project's **settings.py**

Add **CRISPY_TEMPLATE_PACK** to your **settings.py**

In the template, use the crispy filter to render form:

```
<form method="post">
    {% csrf_token %}
    {{ form|crispy }}
    <button type="submit">Submit</button>
</form>
```

Why use Crispy-Forms?

1. Save time on form styling and markup.
2. Keep form code DRY (Don't Repeat Yourself).
3. Easily switch CSS frameworks by changing the template pack.
4. Have consistent, maintainable form layouts.